

Four failure modes, two fixes, and a loop

What happened when we asked 'did the manager name this person?' and the text was Hebrew

CONTENTS

1. The first bug
2. The fix I switched the system off with
3. A note on English, because it is not what I thought
4. What did work, and it is a subtraction
5. The measurement
6. Then someone pointed at what my corpus does not test
7. Failure mode 4: a non-letter inside a word
8. And what actually happened, in production
9. Glitch or attack path
10. And here I have to say something uncomfortable about this article
11. Why this cannot be closed, with the appropriate care
12. "But we have YAP"
13. The idea that looked good, and what happened to it
14. And why it does work in English
15. Hebrew is not a special case
16. Where this sits relative to what is already documented
17. The loop
18. Limitations
19. Appendix: the 89 names run against the three lexicons

We run an engine that does things on behalf of a business manager. It opens tasks, hands responsibility from one employee to another, sends alerts. An action that touches a person is sensitive, so there is a gate: if the model picked a human target that the manager did not name in his instruction, the action does not execute. It goes up for approval.

The idea is simple. Picking a person is the manager's authority, not the model's. The gate only checks whether the manager said the name.

The first implementation was one line:

```
text.includes(name)
```

That held for about two days.

And what I did not know then is that when I tried to count how many times it got it wrong, there would be nothing to count: the gate does not store the text it decided on, so its errors cannot be reconstructed. That is the finding that surprised me most, and it is waiting further down.

1. The first bug

Hebrew glues prepositions and conjunctions onto the front of the word. מש"ה וכל"ב, the mnemonic for the prefix letters: ב, ל, כ, ו, ה, ש, מ. They stack, and "וכשדנה" ("and when Dana") is one legal word. The result is that short names get swallowed inside everyday words all the time.

"שי" (Shai) sits inside "השייכות" (the belonging). "אור" (Or) inside "האור" (the light). "גל" (Gal) inside "בגלל" (because of).

What turned this from a glitch into a bug is the direction. Usually when a filter matches too much, it blocks too much, and that is annoying and visible and someone complains. Here the match does the opposite: it tells the gate "yes, the manager named this person," and the write skips the approval card and executes on someone the manager never mentioned.

The gate meant to stop things opened.

2. The fix I switched the system off with

The next step is obvious to anyone who has ever written a regular expression. You move to a word boundary:

```
new RegExp(`\\b${name}\\b`).test(text)
```

And one line about that code before we go on, because it is wrong in a second way too: the name is interpolated into the expression as-is. A name containing a metacharacter — a dot, brackets, an asterisk — widens the gate or crashes it with an exception. It has to be escaped first, and that holds even when the names come from an employee list and not from user input:

```
const esc = s => s.replace(/[\.\*\?\^\$\{\}\|\[\]\\\]/g, '\\$&')
new RegExp(`\\b${esc(name)}\\b`).test(text)
```

In Hebrew this returns `false`. Always. On every input. With the `u` flag, with the `v` flag, and without them.

The reason is documented, and the moment you see it, it is obvious. `\b` is not a "word boundary" in the linguistic sense, it is a transition between a `\w` character and a character that is not `\w`. And `\w` in JavaScript is `A-Z`, `a-z`, `0-9` and underscore. A Hebrew letter is not `\w`, so the position adjacent to it depends entirely on the other side, and in Hebrew text the other side is a space or a comma. Neither is `\w`. No transition, no boundary, no match.

And this is no longer a bug in the same direction. A gate that always returns `false`, in a polar design like ours that says "not named, therefore escalate," means every action goes up for approval. We moved from a silent failure to a loud one.

This is the skeleton of the article: **two opposite failure directions, and the standard fix for one produces the other.**

3. A note on English, because it is not what I thought

I was sure the story was "the bug is invisible in English." It is not. Substring matching breaks in English just fine:

Name	Text	<code>includes</code>	<code>\b</code>
mark	this is a market problem	✓	✗
bill	the billing page is broken	✓	✗
ann	the announcement goes out	✓	✗
sam	send the same file	✓	✗
ron	that assumption is wrong	✓	✗
ian	the deviant behaviour persists	✓	✗

What is invisible in English is **the failure of the fix**. An English-speaking developer sees `mark`/`market`, adds `\b`, the tests pass, he moves on. In Hebrew that exact same fix does not narrow the gate, it switches it off, and there is no sign of that in the tests he wrote.

4. What did work, and it is a subtraction

What held in the end is tokenization with prefix stripping: you split the text into words, and allow up to three prefixes on the first token of the name.

The detail that looked to me like it carried most of the weight is what was taken out of the list, not what went into it. The definite article ה is excluded. Later in this section I measure that and take this phrasing back.

The reasoning is grammar, and it is not mine. The definite article ה does not attach to a given name, because a given name is definite by its very nature. Gesenius states this in §125. Nobody writes "הדנה" ("the Dana") about a person. Ordinary nouns, by contrast, are very often exactly a name plus "האור" (the light) is not "אור" (Or), "השני" (the second) is not "שני" (Shani), "הקרן" (the fund) is not "קרן" (Keren). In other words, ה is the only prefix whose presence before a name is evidence that this is not the name.

And here I should have stopped and asked how often that evidence is present in writing at all. ה is absorbed after כ, ל, ב. You write "לבית" ("to the house") and not "להבית", and the definite "לגפן" ("to the vine") is written exactly like the indefinite "לגפן" ("to a vine"). So in the three most common definite contexts, the evidence the exclusion is looking for is erased from the surface form.

I measured this against the implementation itself, on five names that are also nouns:

of 5	exclusion works 0	השרון	הקרן	התום	האור	הגפן
of 15	exclusion blind 15	...	באור	באור	בגפן	בגפן

ה is absorbed only after כ, ל, ב; after ו, ש, מ, it stays visible — מהגפן, שהגפן, והגפן are all correctly rejected. Of the twenty prefix sequences in the appendix, eight preserve the evidence and twelve are blind to it, that is sixty percent. And when you look at the most common forms I found in production, לגפן 27 times and לרינה 28, both sit exactly inside the blind range. These two facts sat side by side in front of me for a long time without my ever connecting them. And בשרון too, one of the three false positives I have left, is ביה+שרון — and "השרון" (the Sharon) is exactly one of the geographic exceptions I list in the next paragraph.

This does not destroy the method, and it is still the best of the ones I tested. But "the detail that carried most of the weight" is a phrasing I take back: the exclusion removes one class of false positives, and it is not the large one. The floor I described is higher than I thought.

Three caveats I would rather state myself. The rule holds for names of people; geographic names do take — ה הירדן (the Jordan), הגלבע (the Gilboa), השרון (the Sharon) — and Gesenius lists exactly this exception; the examples he himself gives are הגבעה (the hill), הלבנון (the Lebanon), היאור (the Nile) and הירדן (the Jordan). In informal speech people do say "הדנה הזאת מהחשבונות" ("that Dana from accounting"), and that is exactly the register a manager types in. And a ceiling of three prefixes misses four, so "ולכשדנה" ("and when Dana", four prefixes) is rejected. In all three the failure adds an unnecessary approval card rather than letting an action out, and that is the direction I am willing to pay in.

5. The measurement

Corpus A, eight sentences in which the name is swallowed inside another word and does not refer to a person:

I moved the task because of its project affiliation	העברתי את המשימה בגלל השייכות לפרויקט	שי
someone has to check the light in the office	צריך לבדוק את האור במשרד	אור
the second client in line is waiting	הלקוח השני בתור מחכה	שני
the new fund was approved yesterday	הקרן החדשה אושרה אתמול	קרן
because of the delay we did not finish	בגלל העיכוב לא הספקנו	גל
there is a problem in the northern Sharon today	יש בעיה בשרון הצפוני היום	רון
at the end of the day we will close the report	בתום היום נסגור את הדוח	תום
in light of this we carry on as usual	לאור הדברים נמשיך כרגיל	אור

And eight real mentions. Three carry a prefix on the name, five are bare:

pass the file to Shai	תעביר לשי את הקובץ	שי
and send Dana a reminder	ולדנה תשלח תזכורת	דנה
give Gal this task	תן לגל את המשימה הזו	גל
ask Or what happened	תשאל את אור מה קרה	אור
Shai finished the check	שי סיים את הבדיקה	שי
Or needs to finish the task	אור צריכה לסיים את המשימה	אור
Dana is handling it	דנה מטפלת בזה	דנה
Gal and I will close this	גל ואני נסגור את זה	גל

The seven methods in the table are `includes()`, four boundary variants, the prefix stripping I just described, and the hybrid. "Hybrid" is a single expression that joins the last two: a `\p{L}` boundary on both sides, and inside it up to three prefixes before the name.

```
new RegExp(`(?

```

Method	False positives	True detections
<code>includes()</code>	8 of 8	8 of 8
<code>\b</code>	0	0
<code>\b</code> with <code>u</code>	0	0
<code>\b</code> with <code>v</code>	0	0
<code>\p{L}</code> boundary	0	5 of 8
prefix stripping	3	8 of 8
hybrid	3	8 of 8

The three false positives are `לאור` ← `ריון` ← `בשרון`, `תום`, `בתום`, `אור`. I built a third method to try to close them and got exactly the same three, and a reviewer who built a corpus of his own arrived at the same three.

In each of them, the prefix and the name together spell an existing Hebrew word. And everything that did not fire is the opposite case: "גל" does not match "בגלל", because the stripping stops at "גלל" and never reaches "גל".

Worth saying here that I am not the first to run into this. Itamar Syn-Hershko described the problem of the Hebrew function letters back in 2010, in his work on HebMorph, and showed how naive matching breaks on them. The difference is that for him it was an information retrieval problem, a missing or spurious search result, and for me it is a decision about whether a person stays in the loop.

At this point I thought I was done.

6. Then someone pointed at what my corpus does not test

All eight sentences in Corpus A carry a prefix. Eight out of eight. I built a corpus that tests exactly one failure mode, and I did not notice until a reviewer told me.

And the second mode is the majority, because the Israeli naming convention builds names out of words. אור, שי, גל, תום, קרן, שחר, רון, שני, יעל, תמר, ורד, שקד, אלון, ארז, טל, ברק, אביב, עדן, נועם, שירה, ים, דרור, חן (light, gift, wave, integrity, fund, dawn, joy, second, ibex, palm, rose, almond, oak, cedar, dew, lightning, spring, eden, pleasantness, song, sea, freedom, grace). These are names, and these are words, and there is nothing separating them.

Corpus B, nine sentences in which the name appears as a bare token and does not refer to a person:

we got a green light from the client	קיבלנו אור ירוק מהלקוח	אור
he brought a gift for the holiday	הוא הביא שי לכבוד החג	שי
a wave of complaints came in today	גל של תלונות הגיע היום	גל
at dawn we set out	עם שחר נצא לדרך	שחר
an ibex climbed the rock	יעל טיפסה על הסלע	יעל
a ray of sun came through	קרן השמש חדרה פנימה	קרן
morning dew on the grass	טל של בוקר על העשב	טל
lightning lit up the sky	ברק האיר את השמיים	ברק
a sea of work is waiting for us	ים של עבודה מחכה לנו	ים

Method	False positives out of 9
<code>includes()</code>	9
<code>\p{L}</code> boundary	9
prefix stripping	9
hybrid	9
<code>\b</code>	0

Prefix stripping fails on every one of the rows. So does the hybrid, and so does `\p{L}`. The only method that passes is `\b`, and only because it never matches anything, ever, and that is not a success.

The difference between the two modes is categorical, not quantitative. A prefix adds characters, so it can be stripped. A bare homonym adds nothing. There is nothing to strip, no boundary to fix, no manipulation of

the string that will touch it, because it is identical to the name, character for character.

7. Failure mode 4: a non-letter inside a word

I did not find this one myself. A reviewer ran the code and asked what happens when a character that is not a letter sits inside a word — a vowel point, a hyphen, or an invisible character. The answer takes down the method I chose:

prefix stripping: MATCH	(vowel point)	"שִׁירִי המלאי"	שִׁירִי
prefix stripping: MATCH		"אור<ZWSP>ות המשרד"	אור
prefix stripping: MATCH	(hyphen)	"מערכת אור-קולית"	אור
prefix stripping: MATCH		"תום<RLM>ה נשלחה"	תום

Six out of six that I tested, in prefix stripping, in `\p{L}` and in the hybrid alike. **And the mechanism in the peeler is worse, not the same:** the tokenizer splits on `[^\p{L}]+`, so an invisible character inside a word **creates a new token that is exactly equal to the name**. That is not a partial match, it is a perfect match on a token that was not there.

And this is the same shape for the third time. `\b` died because `\w` is ASCII. The spell checker died because there is no capitalization. And `\p{L}` dies because "a word is a run of letters" — an assumption that holds for the Latin script and not for a script that marks vowels with separate characters. Three layers, one assumption.

The serious consequence is not the direction but the defence. In the attack chapter I lean on the event builders not inserting third-party text. That does not protect here at all, because the manager himself pastes text from email, from WhatsApp and from forms, and pasted text carries ZWSP, soft hyphens, directional marks and vowel points as a matter of routine. And unlike "לאור הדברים", which requires a plausible sentence, an invisible character next to an employee name requires nothing at all and is not visible in the rendered text.

And how many more of these are there. I went looking for a fifth mode myself, and found five:

presentation form (U+FB21)	אור	no match
Hebrew geresh (U+05F3)	ג'ן	no match
gershayim inside a token	ש"י	no match
Latin "o" swapped in	אסא	no match
vowel point in the roster	אור	no match

All of them fail in the closed direction — no match, therefore escalation. Mode 4 is the exception, and the only one that fails open.

And that gives a stopping criterion instead of a chase: **a failure mode enters the security discussion if and only if it fails open.** That class is small and characterizable, because it requires the manipulation to **create** a match — far harder than breaking one. The rest, the ones that fail closed, are not a hole but a load: if an attacker can trigger them at will he produces mass escalation on demand, and that is exactly the T1621 I cite in the loop chapter.

I am not claiming the list is closed. I am claiming that this tail is not a gap in the article but its thesis: a string predicate over a morphologically rich language has no failure list that closes, it has a distribution you sort by direction. And that is precisely why the answer is a structural anchor and not a better regular expression.

8. And what actually happened, in production

One sentence before the numbers, about what they really are. What I call production here is our live database, but almost all of it is our own business: of the 766 texts, 764 are from there and only two from a customer. And the employee list in it was seeded by a seed script, not generated from a natural distribution of names.

Up to here every number in the article comes from a corpus I built by hand. We have logs, so I tried to measure the false positive rate in production.

I could not, and the reason is worth more than the number I was after.

I went looking for the instruction text the gate checks against. It is not stored. It has no column, no run step holds it, and no record contains it. The only place it survives is the rationale of an approval card.

And there is the problem. **An approval card is created only when the gate escalated.** And a false positive, by definition, is the case where the gate said "named," did not escalate, and the write went through. So in the only place the instruction text is kept, a false positive cannot appear.

In this gate, false positives cannot be reconstructed without the input text. Not rare, not zero — there is simply nothing to reconstruct them from. The write itself is logged, so you can cross person-writes against approval cards and manually review what was written without a card; that does not work at scale, but it is not nothing. And this is an observability failure I did not know about until I tried to measure.

What I could check is the list itself, and that surprised me too. I scanned 766 texts in the window between 3.7 and 17.8 (day.month) — 279 card rationales and 487 run summaries — against the 24 given names of the active employees, with the comparison done within each business separately. And I found that **three of the names on our list are ordinary Hebrew words.** I am replacing them here with three fictitious names that carry exactly the same property, and the counts themselves are real: רינה (Rina) is a song of joy, עופר (Ofer) is a young stag, and גפן (Gefen) is the vine.

And because that list was seeded by a script, "three out of 24" here is a statement about whoever wrote the seed and not about how Israelis name their children. I bring it as a sample of the phenomenon, not as a rate.

And the colliding forms show up, again and again:

		4	מגפן	·	28	לרינה	·	27	לגפן	
1	ועופר	·	2	שלרינה	·	4	שגפן	·	4	ורינה

I read them and they look like real mentions of people. But notice what I did in order to know that: **I read the sentence.** I decided by context, which is exactly the thing I argue throughout this article that a token-level gate cannot do. My manual review succeeded where the gate was blind, and that is a live demonstration of the gap between the two solution classes, not evidence of cleanliness.

Alongside the collisions through a prefix sits the bare collision, and that is the larger of the two: 451 matches on a bare name. Of those, the 157 matches on the three homonym names were checked by hand; the remaining 294, on names that are not Hebrew words, were not checked — because there a bare collision is not possible in the first place. I went through the 143 unique context windows of the 157, and all of them are mentions of a person — not one was the noun. Except that 441 out of 451 sit in run summaries, that is, in text the engine itself wrote about its own employees, and in text like that a person-mention is nearly guaranteed in advance. This is not a refutation of the bare collision, it is a measurement whose sampling frame nearly guarantees its own result.

So why did this not blow up. **Not because of the name list, which does collide.** Because of the domain vocabulary. A task-management corpus does not produce sentences about songs of joy, about stags, or about vines. לרינה in our context is almost always a person, because nothing else is being talked about.

And that is a more troubling conclusion than "our names happen to be fine." **The exposure is the product of the name list and the domain vocabulary, and it is invisible until one of the two changes.** The same gate, the same employee list, at a production company or a farm or a plumbing business, and collisions start. Without a single line of code changing, and without anyone knowing that something moved.

This is our loop in a third version: the floor is always there, and what hides it is a lexical coincidence that nobody chose, nobody monitors, and that cannot be seen in the logs.

And from there the practical fix, and it is one column. Store the input the gate decided on — or, if the full text is too heavy, the string that matched and its position inside it — on every decision, and not only when the gate escalated. That is the whole difference between false positives you cannot count and false positives that will be counted tomorrow.

9. Glitch or attack path

This is the first question a security reader will ask, and it determines whether you have just read a correctness bug with a security flavour or a privilege bypass. The answer depends on one thing: **who writes the text the gate checks against.**

For us that text is the manager's instruction, and next to it the event context when the run is autonomous. Two things limit the exposure. First, the event context goes through a sanitizer, meaning it is marked as untrusted input even before this discussion. And second, our event builders are written so that they do not put customer-written text into it: a billing-failure event looks like "חיוב על סך 49.00 ₪ נכשל. סיבה: כרטיס חסום" ("a charge of 49.00 ₪ failed. Reason: card blocked"), without the customer's name, and there are tests that fail if someone adds it.

So for us, today, the three mode 1 false positives are a glitch. I did not demonstrate an exploit, and I am not claiming one.

For mode 4 that does not apply. There the defence does not depend on the event builders at all, because the manager pastes text from outside, and pasted text carries invisible characters routinely. I did not measure that, and I do not know how many of our instructions were pasted.

But notice what protects us, and what gets lost along the way. The sanitizer marks the event context as untrusted input, and that is correct. Except that the gate receives both sources **flattened into a single string**, the manager's instruction and the event context joined by a newline, and at that point the marking no longer exists. The gate does not know which part of the text is trusted and which is not; it looks for a name in a string.

So what protects us is not the gate but the fact that the event builders chose not to include free text written by a third party. That is a content decision, and it can change in one line, without anyone touching the gate.

And from there the general rule, which holds for any system and not only ours: **the moment the authorization gate checks text the system did not compose** — a task description an employee wrote, an incoming email body, a message from a customer, a free-text field in a form — the collision stops being accidental. Someone who knows the organization has an employee named אור (Or) can write "לאור הדברים" ("in light of this") in a free-text field, and cause a write that should have gone up for approval to execute without it.

This does not require special knowledge. It requires knowing one employee's name, and that is information every customer is exposed to.

And the precondition should be called by its name. Third-party text that reaches an agent's context is prompt injection, and that is a bigger vulnerability than anything I have described here. The gate is not the defence against it. It is a downstream mitigation, which narrows what a successful injection can cause. Anyone who

has reached a state where external text enters the instruction the agent acts on has a bigger problem than string matching, and this article deals with the second layer only.

And the blast radius is worth a line, because without it you cannot prioritize. One write that slipped past the gate here is a task opened on a person, a responsibility handed over, or an alert sent. It is logged, it is reversible, it does not touch money and does not send data out. This is not a disaster; it is a decision about a person made without the person who was supposed to approve it. In a different deployment of the same gate, the blast radius will be set by what the tools there can do, and not by what the gate itself does.

10. And here I have to say something uncomfortable about this article

This failure mode is not Hebrew.

I ran it on English and `\b` fired on all twelve rows I tested: Rose in "revenue rose", Mark in "a mark on the wall", Bill in "send me the bill", Grace, Dawn, Will, Art, Hope, Robin, Frank, Summer, Sky. All of them common English names that are also words.

And notice what that does to the table above. There `mark` inside `market` gives a negative `\b`, and that is correct. But `mark` in "he left a mark on the wall" gives a positive `\b`. Same name, same method, opposite result, because the first is the first bug and the second is the third bug.

So `\b` fixes one failure mode, in every language. It fixes nothing else, in any language.

This rearranges the picture, and it is worth being precise about it because the three modes depend on different things. Mode 1 depends on **morphology**: Hebrew glues prefixes, and so does its Latin transliteration, and so does Turkish with its suffixes. Mode 2 depends on **script and engine**: it exists only when the letters are not `\w` in the engine you happen to work in. Mode 3 depends on nothing, it simply exists. And mode 4 depends on **encoding**: it exists in any script that marks vowels or directionality with separate characters.

Four modes, but two directions: 1, 3 and 4 widen the match, 2 narrows it. This is the skeleton I promised at the start, now with the third mode inside it.

And what makes Hebrew distinctive is not that it invents the third one, but that all three fire in it at once. In English mode 1 exists and is easily fixed, and mode 2 does not exist at all, so the problem there is far smaller. But Hebrew is not alone in this — Arabic and Turkish suffer from all three as well, and that is the subject of one of the sections below.

How common is mode 3 in Hebrew? That I did measure, and the number is further down the article: out of 89 Israeli given names I wrote in advance, **40 are also nouns, almost half**. But the more interesting claim is the comparative one, that in Hebrew this is *more* common than in English.

I cannot prove it. I have no parallel English measurement, on a comparable sample and by the same method, and without one there is nothing to compare against. The gap, if it exists, is smaller than my intuition said, so the comparison stays an observation and not a claim.

11. Why this cannot be closed, with the appropriate care

"לאור" the phrase ("in light of") and "לְאֹר" the person ("to Or") are the same sequence of characters. "אֹר" the name and "אור" the glow are the same sequence, without even a prefix separating them. So any matcher that decides on the token alone must give them the same answer, and that is not an observation but a consequence of a deterministic function returning the same value for the same input.

And that does not mean they cannot be told apart. Context tells them apart perfectly well:

followed by an adjective	קיבלנו אור ירוק מהלקוח
"את" followed by a verb, preceded by	תשאל את אור מה קרה

Anyone who reads the full sentence, or holds a lexicon, or runs part-of-speech tagging, will tell them apart. That is exactly what YAP exists for: the morphology and syntax parser from the ONLP Lab, which grew out of the work of Amir More and Reut Tsarfaty. It does lexicon-based morphological analysis followed by joint disambiguation of morphology and syntax, which is exactly the decision a token-level matcher cannot make.

The precise claim is that a token-level matcher has an error floor that cannot be lowered, and that getting off it requires moving to a different class of solution, one that uses context or a dictionary. And that is exactly the point, because an authorization gate is almost always built as a token-level matcher.

12. "But we have YAP"

The objection is fair, and morphological disambiguation really does solve "אֹר" against "אור". And it should be said that this work is good: Tsarfaty and her colleagues wrote explicitly about what is wrong with Hebrew NLP and how to make it right, and tools like YAP and the Elasticsearch Hebrew plugin exist precisely because Hebrew does not yield to naive tokenization. I am not discovering this here, I am running into it from a different direction.

My instinctive objection was that it is "probabilistic" — that you get a confidence score and a threshold out of it, and that this threshold is a security parameter nobody calibrates. I went and checked before writing it down, and it is not true. YAP's documented output format is ten CoNLL columns — ID, FORM, LEMMA, CPOSTAG, POSTAG, FEATS, HEAD, DEPREL, PHEAD, PDEPREL — and none of them is a score, and there

is no threshold in the documentation. There is nothing to calibrate there, because no threshold and no score is exposed. I nearly attacked a mechanism that does not exist.

The real difference is not determinism but the kind of knowledge you have about the failure. With a rule you can enumerate the failure set in advance, and that is what I did: the three false positives in the table above are the entire list for mode 1, and they can be read and reviewed. But that holds only there: the mode 4 class, a non-letter inside a word, is not enumerable — any invisible character, in any position, in any name. So even the rule's promise, that you can enumerate its failures, holds only in part. With a learned model there is an empirical error rate that has to be measured, it is not zero, and you do not know in advance which sentences fall into it.

And it is still the better direction, provided you measure. The problem does not disappear; it moves from a place you can enumerate to a place you have to measure.

13. The idea that looked good, and what happened to it

The token-level floor can be enumerated in advance, and that sounds like an elegant way out.

For every name in the employee list, and for every combination of zero to three prefixes, you ask one question: is the result an existing word. That is a finite computation, a tiny one, and it runs once, offline. Then you route: a safe name goes through the cheap matcher, a flagged name is always escalated to approval, without trying to decide.

And the question "is this an existing word" looks like a dictionary membership question, which is exactly what a spellchecker does. So I ran it on eighty-nine Israeli given names that I wrote before I saw any result, against two independent lexicons: the first is hspell itself, by Nadav Har'El and Dan Kenigsberg, and the second is Word's Hebrew lexicon (MSSP7HE.LEX). The list is published with the article.

Word flagged 89 out of 89. hspell flagged 88 out of 89. Not most of the list, but all of it except one name: "מיטל" (Meital), which is not in hspell's dictionary at all.

Four rows from the list, with every form Word accepted out of the twenty legal prefix sequences listed in the appendix:

אור, באור, באור, לאור, מאור, שאור, שבאור, ...	flagged	20/20	אור
דנה, בדנה, בדנה, לדנה, מדנה, שדנה, שבדנה, ...	flagged	20/20	דנה
אברהם, נאברהם, באברהם, לאברהם, מאברהם, שאברהם, ...	flagged	20/20	אברהם
דניאל, בדניאל, בדניאל, לדניאל, מדניאל, שדניאל, ...	flagged	20/20	דניאל

Look at the last two rows. "אברהם" (Abraham) and "דניאל" (Daniel) are not nouns, they are not supposed to collide with anything, and they were flagged exactly like "אור".

The reason is not that the names are bad but that I picked the wrong tool. **A spellchecker dictionary contains given names on purpose**, otherwise it would underline every name in every document in red. I checked in both lexicons: sixteen biblical names that are not words were found in both, sixteen out of sixteen. And neither of them accepts everything, because out of eight strings I invented in the shape of a name, Word rejected seven and hspell rejected all eight. So the tool distinguishes a word from an arbitrary string perfectly well; it just does not distinguish a word from a name.

And in parallel it strips prefixes on purpose, because "ולאברהם" ("and to Abraham") is correct Hebrew. So it accepts "לאברהם" and rejects "לרגליב". Both behaviours are correct for it, and the two of them together destroy the enumeration: every real name is in the dictionary, and every inflected form of it is in there too.

So a spellchecker is not the oracle the enumeration needs. It answers "is this spellable." The enumeration asks "is this also a noun." In English those are almost the same question, because capitalization separates them; in Hebrew they are different questions, and the dictionary does not tell them apart.

And here I nearly made a second mistake, in the opposite direction. The instinctive assumption is that the right oracle does not exist off the shelf, and it is an easy generalization to make and not check. I checked, and it does exist: there are Hebrew word lists in which nouns and given names are separate files, and then the question stops being a spelling question and becomes a set intersection.

I ran the 89 names against a list like that, 158,346 nouns against 116,669 given names, after six sanity checks that all passed:

noun, not a given name	(table, computer)	שולחן, מחשב
given name, not a noun	(Abraham, Daniel)	אברהם, דניאל
neither	(invented strings)	רגליב, נפתוק

40 of the 89 names are also nouns. 44.9%.

And that changes the conclusion of this section, but not its direction. With the right oracle the enumeration flags 45% of the list instead of 99%, and that is a real improvement. Except that a rule saying "flagged, therefore escalate" still escalates almost half the actions unconditionally, and that is still the second failure direction. The solution I proposed against the first converges on the second, the same pattern, one floor up — only now it has a number instead of an estimate.

And one thing came out of hspell that I did not expect. In its dictionary the prefixes are derived from inflection rules and are not entries in their own right, so you can ask a narrower question: which prefix-plus-

name combinations are themselves a separate dictionary entry, that is, a completely different Hebrew word. The answer, for four of the names in the table:

אור	<-	מאור, שאור	תום	<-	בתום, שתום, שמתום
גל	<-	מגל	דנה	<-	לדנה

"מאור" is a light-giving body, "שאור" is leaven, "בתום" is a colour (orange), "מגל" is a harvesting tool (a sickle). That is failure mode 1 exactly, documented this time from an external lexicon I did not build, and not from a corpus I wrote myself in order to demonstrate it.

14. And why it does work in English

I ran the same twenty English names in lowercase, and got a real distinction:

accepted in lowercase (name is also a word)	13/20	rose, mark, bill, grace, dawn, will, art, hope, robin, frank, summer, sky, melissa
rejected in lowercase (name only)	7/20	jennifer, daniel, abraham, deborah, kevin, brian, samantha

melissa is the only item on the list that is not everyday English; it is the name of a plant, lemon balm. The other twelve are completely ordinary, and without it the ratio is still 12 out of 19 against 7.

Jennifer capitalized is accepted, **jennifer** in lowercase is rejected, and **rose** in lowercase is accepted. English orthography marks given names with a capital letter, so the lexicon separates the two categories and you can ask it the question. Hebrew has no capital letters, the two categories sit in the same entry, and the question cannot be asked.

And this is exactly the same shape we saw in mode 2. There the standard fix was **\b**, and it dies in Hebrew because **\w** is ASCII. Here the standard oracle is a spellchecker, and it dies in Hebrew because there is no capitalization. Twice, at two different layers of the solution, a mechanism quietly leaning on a property of the Latin script.

And that same 44.9% is also the answer to the question I left open earlier, when I wrote that I estimate mode 3 is more common in Hebrew and cannot prove it. Now I can, at least on this sample: almost half the names in the sample are also nouns. The sample was written before I saw any lexicon, so it was not chosen in order to produce this number.

And I want it to be clear that this is not a property of Microsoft. hspell, by Nadav Har'El and Dan Kenigsberg, is a free project built up over years and has nothing to do with the tool I tested against first. It has 469,730 entries, and all sixteen biblical names are in it. Two lexicons built separately, by different people, arrive at the same conclusion.

What remains to be qualified is the path to it. hspell does not ship a built word list but generates it at compile time, so I measured against the Myspell dictionary derived from it, the one whose README states explicitly that it was generated from Hspell version 1.4 data. I did not compile hspell itself.

15. Hebrew is not a special case

Arabic behaves almost identically. It has no ASCII character, so `\b` always returns `false`, and the prefixes are the same story: "النور" (the light) against the name "نور" (Nour), and "السعيد" ("to Said") where the ل attaches to the name itself.

Turkish is more interesting, and it is the one that demonstrates mode 2 in its dangerous form. It is written in Latin letters, so `\b` works, but ı, ğ, ş, ç, ö, ü are not `\w`. And standard Turkish separates a suffix from a given name with an apostrophe, and users skip it:

```
\b "barış" in "barış'a gönder"    -> false  is a name
\b "barış" in "barışa gönder"    -> true   is a name, no apostrophe
\b "barış" in "barışma sürecinde" -> true   not a name
\b "güneş" in "güneş'in raporu"  -> false  is a name
```

Look at the rows. `\b` misses the real mention, catches the thing that is not a name, and changes its answer according to a punctuation mark the user may or may not have typed. It is not noisy, it is inverted.

And that is the practical difference from Hebrew. For us mode 2 is uniform, `false` on everything, and so it is caught by the first test. In Turkish it passes a test with Can and with Kaya, breaks on Barış, and flips on the apostrophe. A uniform failure gets caught. An erratic failure ships to production.

16. Where this sits relative to what is already documented

No fact in the previous sections is new, and I would rather say that myself.

That plain `\b` is not sufficient for Unicode is written in the standard itself, and more sharply than I thought at first. UTS #18 describes the naive mechanism, a boundary between a `\w` character and one that is not, and rules on it: `This is not adequate for Unicode regular expressions`. And immediately after it comes

RL1.4, which is part of **the first conformance level**, and requires that `<word_character>` include all the Alphabetic values from the Unicode database, the digits, and also U+200C and U+200D.

So JavaScript's `\b` is not "level 1 only." **It does not meet level 1.**

And the idea that an over-broad allowlist fails open sits in the catalogue. My bug is, literally, a partial string comparison: **CWE-187, Partial String Comparison**, whose description explicitly names a comparison against `such as a substring`. That is `text.includes(name)` exactly, and that is the right abstraction level to map to — CWE-187 is a Variant marked ALLOWED for mapping.

It is worth saying what *not* to map, too. The instinctive candidate is **CWE-697, Incorrect Comparison**, and that is what I reached for first. 697 is a Pillar, and MITRE marks it **DISCOURAGED** explicitly, on the grounds that it is at too high an abstraction level to describe an actual vulnerability. It is right as a family and wrong as a mapping.

In the neighbourhood also sit **CWE-183, Permissive List of Allowed Inputs**, **CWE-625, Permissive Regular Expression**, and **CWE-777, Regular Expression without Anchors**, which states that an unanchored expression in an allowlist context will cause the protection mechanism to fail. 777 and 183 are not hierarchically related; the link runs through 625 and it is a peer relationship.

And the general pattern, a matching bug that turns into an authorization decision, has a CVE scored 9.8 by NVD: **CVE-2022-22978** in Spring Security. There the mechanism is `.` against a newline character in servlet containers, not anchoring, so I bring it as a pattern and not as an identical case.

And the one-column advice at the end of the production section is not mine either. Logging every authorization decision, and not only the denials, is a known practice: OPA and Aserto have a feature called decision logs that records the input of every decision, and Cerbos records the same thing in its audit log.

And the loop itself has already been modelled, two months before I wrote this. Turan shows that the human reviewer is endogenous to the system — in his words, `the guard's own escalation policy degrades the very oracle it escalates to` — and measures an optimal escalation rate of 64%, meaning that escalating everything is worse than the optimum, not merely more expensive than it. There are numbers there that I do not have, and they support exactly what I described in the enumeration chapter.

What I did not find written down, not there either, is the other direction. His model is about the actions that were escalated and about the capacity of the person receiving them; it does not touch the question of what is recorded when the gate does **not** escalate. And there, in a polar gate that logs only on escalation, the miss rate is not merely unmeasured — it is unrecordable. In a gate like that a miss is not an audit gap, it is a person taken out of the loop without anyone knowing, and without anything left to know it from.

17. The loop

Now what happened here can be described precisely, and it is not what I thought at the start.

We started at failure mode 1, which is silent and dangerous. We fixed it, and we fixed it too hard, and arrived at failure mode 2, which is loud and annoying. We proposed an enumeration to cut the noise, and found that it flags most of the list and therefore produces more noise.

And a queue that everything is escalated into is not stable. It produces rubber-stamping. A person who gets twenty cards a day still reads them, and that is an inbox and not a fatigue regime; but a gate that escalates everything does not stay at twenty. Alert fatigue is well documented in the security operations centre literature, and the loads there are hundreds and thousands of alerts a day; the degradation mechanism is described there precisely, and people approve by reflex until the gate becomes decorative.

And here precision is needed, because this is the closing thesis and I do not want to stretch it. Mode 2 does not degrade back into mode 1 exactly. A reflexive approval still creates a card: there is an approver's name, there is a timestamp, and there is non-repudiation, and you can go back and see who approved what. In mode 1 there is nothing, because the write never went through the gate at all. The degradation zeroes out the *quality* of the human decision, not its trace. But it does zero out what the gate was meant to achieve, because it was built so that a person decides and not so that a person signs. And this happens through the person and not through the code, without a single line of code changing.

And there is an exact precedent for this in security, not just an analogy. MITRE ATT&CK has a dedicated technique for it, **T1621, Multi-Factor Authentication Request Generation**, and its description names "MFA fatigue" explicitly: the attacker floods the victim's approval queue until the victim finally approves just to make it stop.

That is the exact same loop, only triggered deliberately. Which means that flooding the approval queue is not just a side effect of over-cautious engineering, it is a recognized attack vector in its own right — and a gate that escalates everything supplies its precondition in advance, for free, without anyone having to attack.

This is a loop, not a symmetry. The direction from 1 to 2 runs through an engineer who overreacts. The direction from 2 to 1 runs through a person who wears down. Two completely different mechanisms, and both of them predictable.

And that is why you cannot simply choose between the two failure directions. The only way out is to stop asking the question about a string. Not "does the name appear in the text," but letting the manager pick an entity from a list, and anchoring the action to an ID.

Except that this solves one path only. In the autonomous path there is no manager typing, and no list opening in front of him — there is an event that triggers a run. And that is exactly the path I marked as the

riskier one, where the event context is joined to the instruction into a single string and the sanitizer's marking is erased along the way. There, "let the manager pick an entity" says nothing at all.

What is needed there is for the identifier to be created **upstream**, at the moment the event builders phrase the event, and to be carried through the whole run as a separate field and not as text. Or, if there is no such identifier, for every action that touches a person to be escalated unconditionally on that path. Both options are expensive, and both are better than a gate that looks for a name inside a string nobody signed. That is what we are building now.

18. Limitations

The heaviest claim here is the production scan, so it is the one that needs the strongest fencing. Of the 766 texts, 764 came from one business, ours, whose employee list is a demo team seeded into the database and not a team that grew on its own; the two remaining texts came from one customer business. The only two sources were approval-card rationales and run summaries, the window was six weeks, and of the 451 bare-name matches, 441 were found in text the engine itself wrote. I read the 143 context windows in a window of three tokens on each side, not in the full text.

Every one of these factors biases the result in the same direction, so the zero I reported — zero cases in which the match referred to the noun and not to a person — is not a strong claim. I showed above that a false positive cannot appear in an approval card at all, because a card is created only on escalation. That zero describes the product of a particular name list and a particular domain vocabulary in a particular time window, and not a property of the gate.

Both corpora are small and of my own choosing, twenty-five lines in total, and both are published above so they can be argued with. They demonstrate a mechanism and are not a metric.

The `\b` results are not corpus-sensitive, because they follow from the definition of `\w`. You can verify it in one line: `/\w/u.test("א")` returns `false`.

There is one corner in which `\b` does fire on Hebrew: when an ASCII character is adjacent with no space. `/\bאדנהב/u` returns `true` on `"2נהד1"`, on `"abcנהדabc"` and on `"_נהד_"`. In natural Hebrew text this does not happen, but it exists.

The name enumeration ran against two lexicons and not against a frequency corpus, and the two measurements are not identical in character: with Word I asked a full spellchecker, which applies the inflection rules itself, and with hspell I looked for membership in the entry list. Because the prefixes there are derived by .aff rules and are not entries, the number 88 out of 89 is a floor and a full expansion can only add to it. In exactly the same direction, I tested twenty legal prefix sequences and not the 259 sequences the

stripper accepts in the implementation. On the other hand, the list of 89 names is my own choice; I wrote it before I saw any result, and it is published in the appendix so it can be argued with.

The number 40 out of 89 is a floor too. The noun list I used is not complete: `מִיבֵל` in the sense of a container, `שָׁנִי` in the sense of the colour scarlet, and `רֵוַח` in the sense of friendship are not in it, and all three are Hebrew nouns. So the real overlap between names and words is larger than 44.9%, and I do not know by how much.

By contrast, the classification of the production employee list, "three out of 24 are Hebrew words," was done by eye and not against a lexicon. It was not re-run against the dictionary, and it stays an estimate.

And both corpora were written without vowel points and without invisible characters. The true-detection column, 8 out of 8, depends on that: a single vowel point inside a real mention takes down **every** method, `includes()` included, and the result is an escalation. So the side that looks clean in the table was measured on clean text.

And the π -exclusion measurement ran on five names only, chosen because they are also nouns. 0 of 5 against 15 of 15 is a gap too sharp to be chance, but it demonstrates a mechanism and does not estimate a frequency. How many of the actual mentions are definite — that I did not measure.

I measured one engine, JavaScript's. The failure is not a failure of regular expressions but of engines in which `\w` stays ASCII, among them JavaScript, PCRE without `PCRE_UCP`, Java without `UNICODE_CHARACTER_CLASS`, and Go. In Python 3 and .NET it works; John D. Cook has a post that demonstrates this in Python. It is worth being precise about it: his sentence that the metacharacters `work as we would hope` is said about Greek and comes with a caveat, `at least if they're Greek letters`, not about Hebrew. In that same post he does show a Hebrew trap of his own — vowel points that sit between the letters and break a simple match. This table is taken from the documentation and not from a measurement of my own.

And I did not test `Intl.Segmenter`, which is MDN's recommendation for language-aware boundaries. That is the first test I would run next.

19. Appendix: the 89 names run against the three lexicons

The list was written before any run, in two blocks, and without checking a single name. The three runs:

Word spellchecker	89 of 89 flagged
hspell (he_IL dictionary)	88 of 89 flagged
noun list	40 of 89 are also nouns

אברהם, יצחק, יעקב, משה, אהרן, דוד, שלמה, שמואל, יוסף, בנימין, יהודה, שמעון, ראובן, אליהו, יהושע, מרדכי, גדעון, יונתן, נתן, איתן, עומר, גיא, רועי, אורי, נדב, אסף, אלון, תומר, ניר, עידו, רון, שחר, יובל, אמיר, ליאור, ארז, עמית, דור, נועם, אביב, איתן, דניאל, גבריאל, רפאל, מיכאל, אריאל, אדם, יאיר, בועז, עידן, שרה, רבקה, רחל, לאה, מרים, אסתר, רות, נעמי, חנה, דבורה, יעל, תמר, אביגיל, נועה, שירה, מיכל, טל, אור, הדס, עדן, רוני, מאיה, אלה, שקד, ורד, קרן, סיון, הילה, דנה, אורלי, סיגל, מיטל, ליאת, שני, אפרת, גלית, סמדר, רותם, אביטל

The twenty prefix sequences tested for each name are all the legal combinations of ψ , γ , and one of μ , λ , β , κ , including the empty sequence. Explicitly, so that the denominator is reproducible:

ב ב ל מ ש שב שב של שמ (empty)
ו וב ול ומ וש ושב ושל ושם

Sources

Gesenius' Hebrew Grammar §125 —

https://en.wikisource.org/wiki/Gesenius%27_Hebrew_Grammar/125._Determination_of_Nouns_in_general._Determination_of_Proper_Names

MDN, Word boundary assertion — [https://developer.mozilla.org/en-](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/prototype/trimStart)

US/docs/Web/JavaScript/Reference/Regular_expressions/Word_boundary_assertion

UTS #18, Unicode Regular Expressions — <https://www.unicode.org/reports/tr18/>

John D. Cook, Searching Greek and Hebrew with regular expressions —

<https://www.johndcook.com/blog/2020/09/20/unicode-regex/>

Itamar Syn-Hershko, Challenges with indexing Hebrew texts (HebMorph, 2010) –

<https://code972.com/blog/2010/05/challenges-with-indexing-hebrew-texts-hebmorph-part-1-18>

More, Seker, Basmova, Tsarfaty, Joint Transition-Based Models for Morpho-Syntactic Parsing, TACL 7 (2019), 33-48 —

<https://aclanthology.org/Q19-1003/>

YAP, ONLP Lab — <https://github.com/OnlpLab/yap>

Tsarfaty, Seker, Sadde, Klein, What's Wrong with Hebrew NLP? And How to Make it Right —

<https://arxiv.org/abs/1908.05453>

hspell, Nadav Har'El & Dan Kenigsberg — <https://hspell.sourceforge.net/>

he_IL Myspell dictionary, generated from Hspell 1.4 data —

https://github.com/LibreOffice/dictionaries/tree/master/he_IL

Microsoft Office Hebrew proofing tools (MSSP7HE.LEX) – <https://support.microsoft.com/en-us/office/language-accessory-pack-for-microsoft-365-82ee1236-0f9a-45ee-9c72-05b026ee809f>

Elasticsearch, Hebrew lemmatization plugin — <https://www.elastic.co/search-labs/blog/elasticsearch-lemmatization-hebrew-analyzer>

MDN, Intl.Segmenter — https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter

Tariq, Baruwat Chhetri, Nepal, Paris, Alert Fatigue in Security Operations Centres, ACM Computing Surveys 57(9), 2025 — <https://dl.acm.org/doi/10.1145/3723158>

Turan, Oversight Has a Capacity: Calibrating Agent Guards to a Subjective, Fatiguing Human, arXiv 2606.08919 (2026) — <https://arxiv.org/abs/2606.08919>

MITRE ATT&CK T1621, Multi-Factor Authentication Request Generation — <https://attack.mitre.org/techniques/T1621/>

eyaler, hebrew_wordlists — https://github.com/eyaler/hebrew_wordlists

OPA, Decision Logs — <https://www.openpolicyagent.org/docs/management-decision-logs>

Aserto, Decision Logs — <https://docs.aserto.com/docs/decision-logs-guide/api/overview>

Cerbos, Audit block — <https://docs.cerbos.dev/cerbos/latest/configuration/audit.html>

CWE-187: Partial String Comparison — <https://cwe.mitre.org/data/definitions/187.html>

CWE-697: Incorrect Comparison (Pillar, DISCOURAGED for mapping) — <https://cwe.mitre.org/data/definitions/697.html>

CWE-183: Permissive List of Allowed Inputs — <https://cwe.mitre.org/data/definitions/183.html>

CWE-625: Permissive Regular Expression — <https://cwe.mitre.org/data/definitions/625.html>

CWE-777: Regular Expression without Anchors — <https://cwe.mitre.org/data/definitions/777.html>

CVE-2022-22978 (Spring Security, CVSS 9.8) — <https://nvd.nist.gov/vuln/detail/CVE-2022-22978>